

PYTHON CHEAT SHEET

Quick reference for syntax, collections, functions, OOP & everyday coding — built for fast lookup while you work.

QUICK REFERENCE

VARIABLES & DATA TYPES

Creating values and checking their types

<code>x = 10</code>	integer
<code>price = 9.99</code>	float
<code>name = "Saurabh"</code>	string
<code>active = True</code>	boolean
<code>nothing = None</code>	no value / null-like
<code>type(x)</code>	get the type
<code>x, y = 10, 20</code>	multiple assignment
<code>a, b = b, a</code>	swap values

INPUT & OUTPUT

Basic console interaction

<code>print("Hello")</code>	display output
<code>name = input("Name: ")</code>	read text input
<code>age = int(input())</code>	read integer input
<code>print(f"Age: {age}")</code>	formatted output

OPERATORS

Arithmetic, comparison, logical & membership

<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	arithmetic
<code>//</code> , <code>%</code> , <code>**</code>	floor division, remainder, power
<code>==</code> , <code>!=</code> , <code>></code> , <code><</code> , <code>>=</code> , <code><=</code>	comparisons
<code>and</code> , <code>or</code> , <code>not</code>	logical operators
<code>in</code> , <code>/ not in</code>	membership
<code>is</code> , <code>is not</code>	identity

STRINGS

Text, indexing, slicing & common methods

<code>s = "Python"</code>	create a string
<code>s[0]</code> / <code>s[-1]</code>	first / last character
<code>s[1:4]</code>	slice
<code>s.lower()</code> / <code>s.upper()</code>	change case
<code>s.strip()</code>	remove surrounding whitespace
<code>s.replace("Py", "My")</code>	replace text
<code>s.split(',')</code>	split into list
<code>' '.join(items)</code>	join items

LISTS

Ordered, mutable collections

<code>nums = [1, 2, 3]</code>	create list
<code>nums[0]</code> / <code>nums[-1]</code>	indexing
<code>nums[1:3]</code>	slicing
<code>nums.append(4)</code>	add to end
<code>nums.insert(1, 9)</code>	insert
<code>nums.remove(2)</code>	remove matching value
<code>nums.pop()</code>	remove & return last
<code>nums.sort()</code>	sort in place
<code>len(nums)</code>	number of items

TUPLES, SETS & DICTS

Pick the collection that matches the job

<code>t = (1, 2, 3)</code>	tuple: ordered & immutable
<code>s = {1, 2, 3}</code>	set: unique values
<code>d = {'name': 'Sam'}</code>	dict: key → value
<code>s.add(4)</code>	add to set
<code>d['name']</code>	read dictionary value
<code>d.get('age', 0)</code>	safe lookup
<code>d.keys()</code> / <code>d.values()</code>	view keys / values

CONDITIONALS

Run code based on conditions

<code>if x > 0:</code>	start a condition
<code>elif x == 0:</code>	another condition
<code>else:</code>	fallback case
<code>"yes" if ok else "no"</code>	ternary expression

LOOPS

Repeat work over sequences or while true

<code>for x in items:</code>	iterate
<code>for i in range(5):</code>	0 through 4
<code>while condition:</code>	repeat while true
<code>break</code>	exit loop
<code>continue</code>	skip iteration
<code>enumerate(items)</code>	index + value
<code>zip(a, b)</code>	iterate pairs

FUNCTIONS

Reusable blocks of code

<code>def greet(name):</code>	define function
<code>return f'Hi {name}'</code>	return a result
<code>greet('Sam')</code>	call function
<code>def add(a=1, b=2):</code>	default parameters
<code>def total(*args):</code>	many positional args
<code>def show(**kwargs):</code>	many keyword args
<code>lambda x: x * 2</code>	anonymous function

COMPREHENSIONS

Compact collection-building syntax

<code>[x*x for x in nums]</code>	list comprehension
<code>[x for x in nums if x>0]</code>	filtered list
<code>{x*x for x in nums}</code>	set comprehension
<code>{x: x*x for x in nums}</code>	dict comprehension

EXCEPTIONS

Handle errors gracefully

<code>try:</code>	code that may fail
<code>except ValueError:</code>	specific error
<code>except Exception as e:</code>	general error
<code>else:</code>	runs if no exception
<code>finally:</code>	runs either way
<code>raise ValueError('bad')</code>	raise manually

FILES

Read/write files with automatic closing

<code>with open('data.txt') as f:</code>	open and auto-close
	read all
<code>f.read()</code>	read all
<code>f.readline()</code>	read one line
<code>f.readlines()</code>	read lines as list
<code>with open('x.txt', 'w') as f:</code>	write mode
<code>f.write('Hello')</code>	write text

MODULES & PACKAGES

Reuse code & install external libraries

<code>import math</code>	import module
<code>from math import sqrt</code>	import one item
<code>import numpy as np</code>	alias
<code>if __name__ == '__main__':</code>	direct execution guard
<code>python -m pip install pkg</code>	install package

OBJECT-ORIENTED PYTHON

Build programs around classes and objects

<code>class Person:</code>	define class
<code>def __init__(self, name):</code>	constructor
<code>self.name = name</code>	instance attribute
<code>def greet(self):</code>	instance method
<code>p = Person('Sam')</code>	create object
<code>class Child(Parent):</code>	inherit
<code>super().__init__()</code>	call parent constructor

USEFUL BUILT-INS

Small tools that appear everywhere

<code>len(x)</code>	length
<code>sum(nums)</code>	sum values
<code>min(x)</code> / <code>max(x)</code>	smallest / largest
<code>sorted(x)</code>	sorted copy
<code>range(start, stop, step)</code>	number sequence
<code>any(x)</code> / <code>all(x)</code>	at least one / every truthy
<code>map(fn, items)</code>	transform
<code>filter(fn, items)</code>	keep matches

COMMON PATTERNS

Tiny patterns worth memorizing

<code>if x in items:</code>	membership check
<code>for i,x in enumerate(items):</code>	index + item
<code>for a,b in zip(xs, ys):</code>	pair sequences
<code>d.get(key, default)</code>	safe lookup
<code>items.copy()</code>	shallow copy
<code>isinstance(x, int)</code>	type check

VIRTUAL ENVIRONMENT

Keep project dependencies isolated

<code>python -m venv .venv</code>	create environment
<code>.venv\Scripts\activate</code>	Windows activation
<code>source .venv/bin/activate</code>	macOS/Linux activation
<code>pip install -r requirements.txt</code>	install dependencies
<code>pip freeze > requirements.txt</code>	save dependencies
<code>deactivate</code>	leave environment

GITHUB / PROJECT ESSENTIALS

Useful files and commands for real projects

<code>.gitignore</code>	ignore __pycache__, .venv, secrets
<code>README.md</code>	document project
<code>requirements.txt</code>	record dependencies
<code>python main.py</code>	run script
<code>python --version</code>	check Python version
<code>python -m pip --version</code>	check pip version

Remember: indentation defines blocks · mutable: list / set / dict · immutable: str / tuple / int / float · follow PEP 8 for readable code · always use a virtual environment for projects